

Module 5 Week B — Integration Task: Model Comparison & Decision Memo

This is the Module 5 culminating deliverable. Bring together everything from both weeks — linear models (Week A) and tree-based models (Week B) — into a single reproducible evaluation script. Produce a structured model comparison table, visualizations, an experiment log, and a written decision memo recommending a model for Petra Telecom's churn prediction system.

The key insight this assignment reveals: the top models are **close on PR-AUC**. When aggregate ranking metrics don't cleanly separate models, your decision memo has to go beyond single-metric optimization and justify on operational grounds — threshold choice, recall priorities, calibration quality, and deployment cost.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Learning Objectives

After completing this integration, you will be able to:

- Build a complete model comparison pipeline using `sklearn.pipeline.Pipeline`
- Evaluate multiple model families (linear and tree-based) with 5-fold stratified cross-validation
- Produce a comparison table, precision-recall curves, and calibration plots
- Demonstrate concretely what a tree-based model captures that a linear model cannot
- Structure a reproducible ML workflow with experiment logging and model persistence (`joblib`)
- Write a decision memo that translates model metrics into a business recommendation

Setup

1. **Accept the assignment:** [Accept the Integration 5B assignment](#)
2. Clone your repo and open it in VS Code.
3. The repo contains `data/telecom_churn.csv` — **the same expanded Petra Telecom customer sample from the lab** (~4,500 rows, 14 columns, `churned` target at ~16% positive rate).
4. The `results/` directory exists for your output files.
5. Install dependencies:

```
pip install -r requirements.txt
```

6. **Create a working branch:**

```
git checkout -b integration-5b-model-comparison
```

Tasks

Complete the 9 functions in `model_comparison.py`. The tasks build on each other. Run your script with `python model_comparison.py` and run tests with `pytest tests/ -v`.

Task 1 — Load and preprocess

Implement `load_and_preprocess(filepath, random_state)`:

- Load the CSV from the given filepath
- Select only the `NUMERIC_FEATURES` columns into `X` (defined at the top of the file)
- Use `churned` as the target (`y`)
- Split 80/20 with `stratify=y` and the given `random_state`
- Return `(X_train, X_test, y_train, y_test)`

Task 2 — Define 6 model configurations

Implement `define_models()`:

Define exactly 6 model configurations, each wrapped in an `sklearn.pipeline.Pipeline`. The pattern is deliberate — a `default` vs `class_weight='balanced'` pair at **both** the linear and ensemble family levels:

1. **Dummy** — `DummyClassifier(strategy='most_frequent')` with `'passthrough'` scaler (baseline)
2. **LR_default** — `LogisticRegression(max_iter=1000)` with `StandardScaler`
3. **LR_balanced** — `LogisticRegression(class_weight='balanced', max_iter=1000)` with `StandardScaler`
4. **DT_depth5** — `DecisionTreeClassifier(max_depth=5)` with `'passthrough'` scaler
5. **RF_default** — `RandomForestClassifier(n_estimators=100, max_depth=10)` with `'passthrough'` scaler
6. **RF_balanced** — `RandomForestClassifier(n_estimators=100, max_depth=10, class_weight='balanced')` with `'passthrough'` scaler

Set `random_state=42` on every model that accepts it. LR models need `StandardScaler` because linear models are sensitive to feature scale; tree-based models don't need scaling. Each Pipeline has two steps: `('scaler', ...)` and `('model', ...)`.

Return a dict of `{name: Pipeline}`.

Task 3 — Run 5-fold stratified cross-validation

Implement `run_cv_comparison(models, X, y, n_splits, random_state)`:

- Create a `StratifiedKFold` splitter with `shuffle=True`
- For each model, loop over folds: fit on train, predict on validation, compute 5 metrics:
 - **accuracy** — `accuracy_score`
 - **precision** — `precision_score` (with `zero_division=0`)
 - **recall** — `recall_score` (with `zero_division=0`)
 - **F1** — `f1_score` (with `zero_division=0`)
 - **PR-AUC** — `average_precision_score` using `predict_proba` output (threshold-independent ranking)
- Collect fold scores; compute mean and std for each metric
- Return a `DataFrame` with columns: `model`, `accuracy_mean`, `accuracy_std`, `precision_mean`, `precision_std`, `recall_mean`, `recall_std`, `f1_mean`, `f1_std`, `pr_auc_mean`, `pr_auc_std`

Important: PR-AUC requires probability scores from `predict_proba`, not binary predictions from `predict`. The `DummyClassifier`'s `predict_proba` returns the class prior for every sample — its PR-AUC should equal the positive rate (~0.16).

Task 4 — Save comparison table

Implement `save_comparison_table(results_df, output_path)`:

- Save the results `DataFrame` to CSV at the given path (default: `results/comparison_table.csv`)

Task 5 — PR curves for top 3

Implement `plot_pr_curves_top3(models, X_test, y_test, output_path)`:

- Compute PR-AUC for each fitted model on the test set
- Select the top 3 by PR-AUC
- Plot precision-recall curves for all 3 on the same figure using `PrecisionRecallDisplay.from_estimator`
- Title the plot, save to the output path, close the figure

Task 6 — Calibration curves for top 3

Implement `plot_calibration_top3(models, X_test, y_test, output_path)`:

- Same top 3 models as Task 5
- Plot calibration curves using `CalibrationDisplay.from_estimator` with `n_bins=10`
- Save to the output path, close the figure

Calibration shows whether `predict_proba` outputs correspond to actual positive rates. A well-calibrated model follows the diagonal.

Task 7 — Save the best model

Implement `save_best_model(best_model, output_path)`:

- Save the fitted Pipeline using `from joblib import dump`
- The `main()` function determines which model is "best" (by PR-AUC) and passes it in — your function just saves it

Task 8 — Log experiment results

Implement `log_experiment(results_df, output_path)`:

- Create a log `DataFrame` with columns: `model_name`, `accuracy`, `precision`, `recall`, `f1`, `pr_auc` (use the mean values from `results_df`), and a `timestamp` column with the current time in ISO format
- Save to the output path as CSV

Task 9 — Tree-capability analysis

Implement `find_tree_vs_linear_disagreement(rf_model, lr_model, X_test, y_test, feature_names, min_diff)`:

This is the same tree-vs-linear capability demonstration from the lab, but now using **Pipelines** (so both models accept the same raw `X_test` — the Pipeline handles scaling internally for LR).

- Get `predict_proba` from both pipelines on `X_test`
- Compute the absolute difference of `P(churn=1)`
- Find the sample with the **maximum** difference (must be $\geq \text{min_diff}$, default 0.15)
- Return a dict with: `sample_idx`, `feature_values` (dict of feature name $\rightarrow$ value), `rf_proba`, `lr_proba`, `prob_diff`, `true_label`

The `main()` function writes this to `results/tree_vs_linear_disagreement.md`. You fill in the **Structural Explanation** section: 2–3 sentences explaining WHY these models disagree on this sample. Point to a specific feature interaction, non-monotonic relationship, or threshold effect the tree captured that the linear model could not.

Task 10 — Decision memo (in PR description)

Write a decision memo (3–4 paragraphs) in your PR description. Your memo must include:

1. **Which model you recommend and why.** Name the specific model and cite metrics from your comparison table.
2. **What tradeoffs you made.** Specifically discuss the `class_weight='balanced'` effect at the **default 0.5 threshold**: it shifts the operating point (dramatically increasing recall by trading precision), but it is not a ranking improvement — PR-AUC (which is threshold-independent) changes little. Be explicit: "at the default 0.5 threshold" is required whenever you discuss the `class_weight` effect.
3. **What operational KPIs this model serves.** Map at least one metric to a business outcome:
 - **Customer retention rate** — driven by recall (how many churners does the model catch?)
 - **False alarm cost** — driven by precision (how many offers go to non-churners?)
 - **Early warning lead time** — driven by calibrated probabilities
4. **At least one concrete pattern** that a tree-family model captures that a linear model cannot on this data. Cite your tree-capability analysis from Task 9.
5. **One limitation or risk** of the recommended model (e.g., concept drift, fairness, interpretability).

Submission

Open a pull request from `integration-5b-model-comparison` to `main`. Your PR must include:

1. `model_comparison.py` — complete, runnable script (`python model_comparison.py` produces all outputs)
2. `results/` directory containing:
 - `comparison_table.csv`
 - `pr_curves.png`
 - `calibration.png`
 - `best_model.joblib`
 - `experiment_log.csv`
 - `tree_vs_linear_disagreement.md` (with your structural explanation filled in)
3. PR description with:
 - Model comparison table (all 6 models $\times$ 5 metrics with mean $\pm$ std)
 - Decision memo (3–4 paragraphs addressing all five points above)
4. Paste your PR URL into TalentLMS $\rightarrow$ Module 5 Week B $\rightarrow$ Integration Task to submit this assignment.

Challenge Extensions

These are optional extensions for learners who finish the base requirements. They build toward professional ML engineering practice.

Tier 1 — Threshold Optimization for Deployment

The base task uses the default 0.5 threshold. In production, the threshold is a business decision.

- For the recommended model, sweep thresholds from 0.1 to 0.9 in steps of 0.05
- Compute precision, recall, F1, and the expected number of alerts per 1,000 customers at each threshold
- Petra Telecom's retention team can contact at most 150 customers per month. Given a customer base of 10,000, what threshold achieves this capacity constraint while maximizing recall?
- Add a "Threshold Recommendation" section to your decision memo with the chosen threshold and its operational implications
- Save the threshold sweep plot to `results/threshold_sweep.png`

Tier 2 — Permutation Importance and Model Explanation

Feature importance from random forests (MDI) is biased toward high-cardinality features. Permutation importance provides a model-agnostic alternative.

- Compute permutation importance for your top 3 models using `sklearn.inspection.permutation_importance` (use the test set, 10 repeats)
- Compare the permutation importance rankings across model families (logistic regression vs. random forest). Do they agree on which features matter?
- Create a grouped bar chart showing permutation importance for the top 8 features across all 3 models
- Write a paragraph: if features rank differently across model families, what does that tell you about each model's decision-making process?
- Save the comparison plot to `results/permutation_importance.png`

Tier 3 — Build a Model Selection Framework

The base task hardcodes 6 models. In production, the model list, hyperparameter grids, and evaluation metrics should be configurable.

- Design and implement a `ModelSelector` class that:
 - Accepts a YAML or JSON configuration file defining models, hyperparameter options, and evaluation metrics
 - Runs the full comparison pipeline (cross-validation, comparison table, plots, experiment log) from configuration alone
 - Supports adding new model types without modifying the core code (use scikit-learn's string-based model lookup or a registry pattern)
 - Produces a structured output directory with timestamped experiment folders
- Write a configuration file that reproduces your base task results
- Write a second configuration file that adds `GradientBoostingClassifier` with 3 hyperparameter combinations
- Document the design in a README: what would a teammate need to know to add a new model family?

